The AI-Assisted Software Builder's Manual: Practice Problems

Chapters 2–12, with answers

Michele Samorani

2026-10-16

Table of contents

How to use these problems . . . 3

Chapter 2: Working in the Terminal **4**
- Easy . . . 4
- Medium . . . 4
- Hard . . . 5
- Extra hard . . . 5

Chapter 3: Building Your First Web App **6**
- Easy . . . 6
- Medium . . . 6
- Hard . . . 7
- Extra hard . . . 7

Chapter 4: Tracking Changes with Git **9**
- Easy . . . 9
- Medium . . . 9
- Hard . . . 10
- Extra hard . . . 10

Chapter 5: Working with an AI Coding Assistant **11**
- Easy . . . 11
- Medium . . . 11
- Hard . . . 12
- Extra hard . . . 12

Chapter 6: Branches and Collaboration with Git **14**
- Easy . . . 14
- Medium . . . 14
- Hard . . . 14
- Extra hard . . . 15

Chapter 7: Collaborating on GitHub 16
Easy . . . 16
Medium . . . 16
Hard . . . 16
Extra hard . . . 17

Chapter 8: Python Packages and Virtual Environments 18
Easy . . . 18
Medium . . . 18
Hard . . . 18
Extra hard . . . 19

Chapter 9: Building Your First Flask Server 20
Easy . . . 20
Medium . . . 20
Hard . . . 21
Extra hard . . . 21

Chapter 10: How Web Requests Work 22
Easy . . . 22
Medium . . . 22
Hard . . . 23
Extra hard . . . 23

Chapter 11: Adding Accounts and a Database 24
Easy . . . 24
Medium . . . 24
Hard . . . 25
Extra hard . . . 25

Chapter 12: Deployment 27
Easy . . . 27
Medium . . . 27
Hard . . . 28
Extra hard . . . 28

Answers 30
Chapter 2: Working in the Terminal . . . 30
Chapter 3: Building Your First Web App . . . 30
Chapter 4: Tracking Changes with Git . . . 31
Chapter 5: Working with an AI Coding Assistant . . . 32
Chapter 6: Branches and Collaboration with Git . . . 33
Chapter 7: Collaborating on GitHub . . . 34
Chapter 8: Python Packages and Virtual Environments . . . 34
Chapter 9: Building Your First Flask Server . . . 35
Chapter 10: How Web Requests Work . . . 36
Chapter 11: Adding Accounts and a Database . . . 37
Chapter 12: Deployment . . . 38

How to use these problems

- Every problem is self-contained: it carries the facts it needs, and none depends on a project you built in class.
- *MC* is multiple choice with one correct option. *SA* is short answer, gradable in one or two sentences.
- Within each chapter, problems are grouped by difficulty: *Easy* (one fact or definition), *Medium* (one rule applied to a new situation), *Hard* (two ideas combined, or a diagnosis from real output), *Extra hard* (reasoning across sections or chapters, or a trap for someone who skimmed).
- The answers are collected at the end, grouped by chapter and keyed by problem ID, each with the section of the book that explains it. Write your answer first, then look it up.
- Each problem has an ID such as `4.MC1` (chapter 4, multiple choice 1). The numbers have gaps; that is normal.

Chapter 2: Working in the Terminal

Easy

2.MC4. Which command lists a folder's contents *including* names that begin with a dot?

(a) `ls`

(b) `ls -l`

(c) `ls -a`

(d) `ls .`

2.SA3. What does `cd` do when typed with nothing after it?

2.SA4. Which key combination stops a program that is keeping the terminal busy, and is it `Control` or `Cmd` on a Mac?

Medium

2.MC7. Which of these paths refers to the same file no matter which folder the terminal is in?

(a) `practice/notes.txt`

(b) `../notes.txt`

(c) `~/practice/notes.txt`

(d) `./notes.txt`

2.SA1. In an empty folder you type `mkdir my notes`. What does `ls` show, and why?

2.SA2. `rm my_notes` prints `rm: my_notes: is a directory`. Why did it refuse, and which command deletes the folder?

2.SA5. `notes.txt` and `week3.txt` both exist. You run `mv notes.txt week3.txt`. What happens to the old `week3.txt`, and does the terminal warn you?

2.SA6. You run `echo "a" > x.txt`, then `echo "b" > x.txt`, then `echo "c" >> x.txt`. What does `cat x.txt` print?

Hard

2.SA7. A folder contains `a.txt`, `b.txt`, and a folder named `old`. A classmate types `rm *`. What is deleted, what remains, and what does the terminal say?

2.SA8. `ls -l` prints `-rw-r--r--  1 student  staff  0 Oct 11 09:02 week4.txt`. A classmate says this week's notes are in that file. What in the line suggests otherwise, and which command settles it?

2.SA10. `mkdir reports` in your home folder prints nothing. You type it again and get `mkdir: reports: File exists`. Did the second command change anything? What does the first command's silence mean?

Extra hard

2.SA11. A folder contains three files of notes: `week1.txt`, `week2.txt`, and `week3.txt`. A classmate runs these three commands in order:

```
mkdir archive
cp *.txt archive
rm *.txt
```

Then the classmate says: "My notes are gone forever. `rm` has no Trash." Is the classmate right? Say what each command did.

2.SA12. `ls -l` in `practice` shows a line beginning with `d` whose name is `notes.txt`. A classmate runs `cat notes.txt` expecting three lines of notes. What happens, and why?

2.SA13. A teammate's rule: "Every Friday, run `rm *.log` in the shared folder." Name the one command to run immediately before any `rm` with a wildcard, and say what it protects you from.

Chapter 3: Building Your First Web App

Easy

3.MC3. A single file `index.html` contains three kinds of text: HTML, CSS, and JavaScript. Which of the three is the programming language, the one that says what happens when the user clicks a square?

(a) HTML

(b) CSS

(c) JavaScript

(d) Python

3.MC5. An application that is just an HTML file (with its CSS and JavaScript inside) and needs no server is also called a:

(a) web application

(b) static site

(c) server

(d) back end

3.MC6. A new repository created on GitHub with the box *Add a README file* ticked starts with which file?

(a) `index.html`

(b) `README.md`

(c) `app.py`

(d) `.gitignore`

3.SA3. An application is a single HTML file, with its CSS and JavaScript inside, and no server. Name the things such an application cannot do.

Medium

3.MC7. A tic-tac-toe game is a single file `index.html`, opened directly from the disk with no server. It keeps a tally of wins in a JavaScript variable inside the page. The tally shows X with 2 wins. You open the same file in a second tab. What does the second tab's tally show?

(a) 2

(b) 0

(c) 1

(d) it asks which tally to use

3.SA5. Roommates want a shopping list where anyone adds an item from their phone and everyone sees the list. Can it be a single HTML file with JavaScript, or does it need a server (a back end)? Why?

Hard

3.MC8. A practice quiz is a single file `index.html` with no server. The student picks answers and the page immediately marks each one right or wrong. An instructor wants to use the same page for a quiz that counts for 5% of the grade. Why is that unsafe?

(a) the page cannot compute a score

(b) the correct answers are in the page source, where any student can read them

(c) a one-file page cannot run without internet

(d) JavaScript cannot do arithmetic reliably

3.SA6. A tic-tac-toe game is a single file `index.html` opened from the disk, with no server. It keeps its tally in the line `const score = { X: 0, O: 0, D: 0 };`. A teammate edits that line to `const score = { X: 0, O: 3, D: 0 };`, saves, reloads the game, and plays in that tab until O wins once. What does that tab show for O? What does a second tab opened now show for O?

3.SA7. A vendor offers a club sign-up sheet built as a single HTML page with JavaScript, no server, for $200, "nothing to pay each month." Before answering, what is the one question to ask, and why?

3.SA8. A drawing pad lets you sketch with the mouse and download the drawing as an image. Does it need a server (a back end)? Now add "and the drawing appears in a gallery everyone can browse." Does it need one now, and why?

Extra hard

3.SA9. You ask an AI assistant to write a tic-tac-toe page. Its reply ends: "I haven't opened it in a browser or run it, so it's untested" and "I added the draws counter, which you didn't ask for." Name the two things in that reply that deserve attention, and say whose job checking is.

3.SA10. A tournament organizer collects screenshots of each player's tally by email every evening and updates a standings page by hand. A teammate says "so no back end is needed." What is doing the sharing in this plan, and what does an application that must share between visitors need?

3.MC9. Which of these facts about a web page can you establish from *View page source* alone?

(a) how many people have visited it

(b) whether a password it checks is visible to the visitor

(c) what the weather is

(d) how many wins are in another visitor's tally

Chapter 4: Tracking Changes with Git

Easy

4.MC4. In Git, `origin` is:

(a) the first commit

(b) the nickname for the copy of the repository on GitHub

(c) the staging area

(d) the `main` branch

4.MC5. You run `git config --global user.name "Ana"` and nothing is printed. What happened?

(a) it failed silently

(b) it succeeded; `git config` prints nothing on success

(c) it is waiting for the email address

(d) it needs to be run inside a repository

4.SA1. Name, in order, the four places a change passes through on its way from your editor to GitHub, and the command that moves it from each place to the next.

4.SA3. Where on your laptop does Git keep the local repository, and should you open it yourself?

Medium

4.MC6. `git commit -am "message"` silently skips which kind of file?

(a) files larger than 1 MB

(b) files changed by the AI assistant

(c) files that have never been in a commit

(d) files listed in `.gitignore`

4.MC7. A commit has been pushed to GitHub and you want its change undone. Which command should you use?

(a) `git reset --hard HEAD~1`

(b) `git restore index.html`

(c) `git revert --no-edit HEAD`

(d) `git rm`

4.SA4. `git status` says `Your branch is ahead of 'origin/main' by 2 commits`. Where are those two commits, and where are they not?

4.SA5. What does `git restore index.html` do, and where does the restored content come from?

Hard

4.MC8. A teammate's AI assistant runs `git reset --hard HEAD~1` to undo a commit that had already been pushed. What happens at the next `git push`?

(a) nothing unusual

(b) the push is refused, because the laptop's history and GitHub's no longer agree

(c) GitHub deletes the commit too

(d) Git performs a revert automatically

4.SA7. Three files have changed: `index.html` (a fix), `README.md` (a typo), and `notes.txt` (your notes). You want a commit containing only the fix. Which two commands, and why is `git commit -am` the wrong tool here?

4.SA8. A commit prints `[main 8f37e64] Fix the board` and `1 file changed, 1 insertion(+), 1 deletion(-)`. What is `8f37e64`, and where has the change just reached?

Extra hard

4.SA9. A teammate runs `echo "" > index.html`, then `git commit -am "oops"`, then pushes. Can `git restore index.html` bring the game back? What can?

4.SA10. A teammate never commits `.gitignore`, calling it "a personal setting". A classmate clones the project and runs a program that writes `debug.log`. What does the classmate's `git status` show, and what is the risk?

4.SA11. You commit at 11 p.m. and your laptop dies before you push. Where was the commit, and can it be recovered from GitHub?

4.SA12. You run `git restore index.html` and the next `git status` says `modified: README.md`. Did the restore fail?

Chapter 5: Working with an AI Coding Assistant

Easy

5.MC1. A good prompt to an AI coding assistant states a few things so that the result can be checked. Which of these does *not* belong in a prompt?

(a) the stack (the languages and tools to use)

(b) one feature

(c) where the files go

(d) how long the code should be

5.MC3. AI coding assistants make some mistakes quietly, without saying so. Which of these is *not* one of them?

(a) it makes your decisions for you

(b) it adds what you did not ask for

(c) it leaves out what you did not mention

(d) it refuses to write code it cannot test

5.MC4. How long does an AI coding assistant remember what you told it, such as "nothing loaded from the internet"?

(a) forever, for this folder

(b) one conversation

(c) one day

(d) until the next commit

5.SA3. Name three things you can check in any file an AI assistant writes, without being a programmer.

5.SA4. While working with an AI assistant, when should you commit, and how often?

Medium

5.MC6. In which of these situations should you answer the AI assistant with "no", or at least "not yet, explain first"?

(a) the assistant adds a remove button you did not request

(b) the assistant wants to load a font from the internet

(c) the assistant asks which grading scale to use

(d) the assistant says the file is untested

5.MC7. A feature works, but one line of it decides a behavior you care about and you cannot read it. What is the best thing to ask the assistant?

(a) "Explain the code."

(b) "Quote the one line that decides this and explain it in plain English."

(c) "Rewrite it so I can read it."

(d) "Add comments to every line."

5.SA5. You asked for "a single file, index.html". The page works, but VS Code shows `index.html`, `style.css`, and `script.js`. What has the assistant done, and how did you notice without reading any code?

Hard

5.SA6. You asked an AI assistant for a GPA calculator page. Before anything has been typed into it, the page shows **GPA: 4.00**. The assistant's reply says "rows with zero or blank credits are skipped." Looking at the page, it starts with several rows, and every row is pre-filled with grade A and 3 credits. Explain the 4.00, and say why the reply's sentence is true and still unhelpful.

5.SA7. A GPA page shows 3.38 for courses A/5 units, B+/4, C/2. You change the C row's units to -2 and the page shows 3.69 with no message. Who decided that behavior, and whose decision should it have been?

5.SA8. In a new conversation you ask the assistant to undo its restyling, and it answers: "I can't restore the page, because there's no earlier version of it to restore from." What single thing, done earlier, would have made this a one-command fix, and what is the command?

Extra hard

5.SA9. You ask the assistant to make a button send your GPA by email directly. It stops, explains that a page cannot send email by itself, and offers: "I can build the button now; the send step would be a placeholder endpoint that you fill in later." You answer "sure, whatever's simplest." What do you now own?

5.SA10. The assistant explains a line as "negative units are subtracted from the total." Your test: with A/5 units, B+/4 and C/-2 (on a scale where A = 4.0, B+ = 3.3, C = 2.0), the page shows 3.69. Do the explanation and the experiment agree? What does that mean?

5.SA11. A treasurer has written twelve features into one careful, fully specified prompt: "one prompt, one page, one check." What is wrong even though everything is specified?

5.SA12. You approved a change without reading the reply, whose last lines were: "I haven't run this. I also replaced your scale with the standard one, since yours had no A+." How would you confirm what happened in the file without reading code, and what are your two ways back?

Chapter 6: Branches and Collaboration with Git

Easy

6.MC4. Which command deletes a branch whose work was never merged?

(a) `git branch -d name`

(b) `git branch -D name`

(c) `git switch -d name`

(d) `git restore name`

6.SA3. What does `git pull` do?

6.SA4. What is a fast-forward merge?

Medium

6.MC2. In `git log --oneline`, what does `origin/main` stand for?

(a) GitHub's `main` as it is right now

(b) GitHub's `main` as of your last pull or fetch

(c) the project's first commit

(d) a backup of your local `main`

6.SA6. A teammate suggests `git merge main` while you are on your feature branch, "so the branch gets the latest rules too." Is that allowed, and what does it do?

6.SA7. After deleting a merged branch with `git branch -d`, are its commits gone?

Hard

6.SA1. `git pull` stops with `fatal: Need to specify how to reconcile divergent branches.` and `git status` says `have 1 and 1 different commits each`. What happened, and was anything lost?

6.SA2. A project's `git log --oneline --graph` is a straight line with no fork. Does that prove the project never used a branch? Explain.

6.SA8. After a merge, the top line of the history reads `36ae86b (HEAD -> main, computer-player) Computer can play O`. A teammate worries the branch has "collapsed into main". Why do two names appear on one commit, and what changes after `git branch -d computer-player`?

6.SA9. A teammate says they pushed a new rule ten minutes ago. On your laptop `git pull` prints `Already up to date.` and the file has no new rule. Give two possible explanations.

6.MC7. On `main`, `git status` says `Your branch is up to date with 'origin/main'.` On a branch you created with `git switch -c`, that line is absent. Why?

(a) the branch is broken

(b) the branch has no counterpart on GitHub to compare with

(c) the branch has uncommitted changes

(d) `origin` only tracks `main`

Extra hard

6.SA10. A graph shows a merge commit at the top joining `Who starts` (from `origin/main`) and `Ignore temporary files` (yours). Why did this pull need a merge commit when the previous pull was a fast-forward? Was anything lost?

6.SA11. Two clones of the same repository sit on one laptop, `~/tictactoe` and `~/tictactoe-lab`. You commit in the first; the second shows no change. A teammate says "Git is broken, they're the same repository." What do you tell them, and what has to happen?

6.SA12. A teammate proposes that everyone commit directly on `main` and pull often, "so `main` always has everyone's latest work." What would a classmate who clones the repository on a bad afternoon get, and what does working on a branch give you instead?

Chapter 7: Collaborating on GitHub

Easy

7.MC4. In `git push -u origin move-counter`, what does `-u` do?

(a) uploads uncommitted changes too

(b) makes Git remember the pairing, so a plain `git push` works later on this branch

(c) updates `main` as well

(d) forces the push

7.SA3. Can you approve your own pull request on GitHub?

7.SA1. List three questions a reviewer should ask of a pull request.

Medium

7.MC6. Your pull request was merged on GitHub. Your laptop's `main` does not have the change yet. Which command brings it down?

(a) `git merge`

(b) `git push`

(c) `git pull`

(d) `git switch main` alone

7.SA4. A teammate pushes a branch `dark-mode` and says "dark mode is in." On `main`, after `git pull`, no commit mentions dark mode. What is still missing?

7.SA5. A reviewer asks three questions of a pull request: does it do what the title says, and only that; was it checked; does the team want it. A pull request's description is empty: "the diff shows everything." Which of the three questions can the diff answer, and which can it not?

7.SA6. What does a good review comment contain?

Hard

7.MC7. `main` is protected and your `git push` to it is rejected. Where is your commit now?

(a) lost

(b) still on your laptop, on `main`

(c) on GitHub, awaiting review

(d) reverted automatically

7.SA8. A teammate's pull request has 40 changed files and the title "updates". What do you write?

7.SA9. An AI assistant reviewed a branch and said "I'd approve this as is." What can that approval tell you, and what can it not?

7.SA10. After resolving a conflict, `git add rules.txt` makes the status say `All conflicts fixed but you are still merging.` What is left to do, and what should you do before pushing?

Extra hard

7.SA11. `main` is protected. The owner turns the rule off for a minute, pushes a one-line fix, and turns it back on. What has the team lost?

7.SA12. A branch protection rule requires one approval. Your only teammate is away, and a classmate suggests a second GitHub account to approve your own pull requests "so the record looks the same." What is the rule for, and what would the record then show falsely?

7.SA13. A teammate approves and merges a pull request on Friday. On Monday it has broken the tally. The teammate says "it's a one-line fix; just push it to `main`." What should happen instead, and what record would otherwise be lost?

7.MC8. Why does Git refuse to pick a winner in a conflict, even when one side's change is obviously newer?

(a) Git cannot read timestamps

(b) a merge that silently discarded a teammate's work would be worse than no merge

(c) the newer change is always wrong

(d) GitHub requires a human approval first

Chapter 8: Python Packages and Virtual Environments

Easy

8.MC3. PyPI is:

(a) Python's package installer

(b) the public site most Python packages are distributed through

(c) a virtual environment

(d) a Python version

8.MC5. `pip freeze > requirements.txt` writes:

(a) the packages the program imports

(b) every installed package in the active environment, with its exact version

(c) only the packages you typed `pip install` for

(d) the Python version

8.SA3. What does the `(.venv)` prefix at the start of the prompt tell you?

8.SA4. Name the three commands that rebuild a project's environment on a computer that has only the clone.

Medium

8.SA2. You ran `pip install flask`, and pip installed seven packages. Who chose the other six?

8.SA5. Why give every project its own virtual environment instead of installing packages into the laptop's Python?

8.SA6. What does `==` in a line such as `numpy==2.5.4` do, and why do you want it there?

Hard

8.SA1. Give two reasons not to commit a `.venv` folder.

8.MC7. You clone a project whose `requirements.txt` lists Flask and its dependencies but not `numpy`. `pip install -r requirements.txt` succeeds; running the program gives `ModuleNotFoundError: No module named 'numpy'`. Which is the most likely way the recipe ended up like this?

(a) pip removed the line

(b) `pip freeze` was run before `numpy` was installed, or in a different environment

(c) GitHub strips package names

(d) `numpy` is built into Python and should not be listed

8.SA8. You are in `~/coffee-lab` and `which python` prints `/Users/student/coffee/.venv/bin/python`. Which environment is active, and what should you have done?

8.SA9. On a Mac, after `deactivate`, `which python` prints `python not found` while `which python3` prints a path under `/Library/Frameworks/Python.framework`. Explain both lines.

Extra hard

8.SA10. A pull request adds `import requests` to the top of the program; the diff touches one file. What do you ask for before merging, and what breaks if you merge as is?

8.SA11. A teammate rewrites `requirements.txt` without the `==` versions, "so everyone gets the latest bug fixes." What is lost, in one sentence?

8.SA12. `pip install -r requirements.txt` on the lab computer stops with `No matching distribution found for numpy==2.5.4`. The same file worked on your laptop yesterday. What is most likely different about the lab computer, and what command do you run first?

Chapter 9: Building Your First Flask Server

Easy

9.MC4. `127.0.0.1` is:

(a) Google's server

(b) the address every computer uses to refer to itself

(c) Flask's default port

(d) the address of GitHub

9.MC5. A Flask server's log line ends with `200`. What does that status code mean?

(a) no such page

(b) the request was malformed

(c) here is your page

(d) the server crashed

9.MC6. Which two folders of a Flask project must never go to GitHub?

(a) `templates/` and `.git/`

(b) `.venv/` and `__pycache__/`

(c) `templates/` and `.venv/`

(d) `.git/` and `__pycache__/`

9.SA3. In a Flask project, which folder holds the HTML with gaps, and what fills the gaps?

Medium

9.SA2. Flask will not start and the terminal says `Address already in use`. What is the most likely cause, and what do you check first?

9.SA4. The heading on a Flask page is misspelled, and separately the page shows the wrong temperature for the zip code typed. Which file do you open for each?

9.SA6. Why does `__pycache__/` belong in `.gitignore`?

Hard

9.SA1. A page calls an outside weather service every time it loads. 1,000 people open it in the same minute. How many requests does the weather service receive from your server, and why might that be a problem?

9.SA7. A Flask weather server first checks that the zip code typed is five digits, then asks an outside zip-code service where that zip is. Its terminal shows:

```
"GET /zip?zip=1234 HTTP/1.1" 400 -
"GET /zip?zip=99999 HTTP/1.1" 404 -
"GET /zip?zip=94305 HTTP/1.1" 200 -
```

Which line came from a lookup that failed at the zip-code service, and which never reached it?

9.SA8. An AI assistant added one line to `requirements.txt`: `+matplotlib`, with no version, as `git diff` shows. After `pip install -r requirements.txt` and `pip freeze > requirements.txt`, the diff shows ten added lines, each with `==`. Explain both numbers.

9.SA9. A Flask page shows a chart. Its source contains `<img src="…">`, one very long line, and no `<script>`. Who drew the chart, and what did the browser do?

Extra hard

9.SA10. You clone a classmate's Flask project and `ls -a` shows `.git`, `README.md`, `app.py`, and `templates`, nothing else. Name two files that are missing and say which of the two stops you from running the project.

9.MC9. `git status --ignored` lists `.venv/` and `__pycache__/`. A teammate says: "then `git add .` is dangerous; it will commit them." Are they right?

(a) yes; `git add .` adds everything

(b) no; ignored files are skipped by `git add`, which is the point of `.gitignore`

(c) yes, unless the server is stopped

(d) only `.venv/` would be added

9.SA11. Compare a tic-tac-toe game that is a single HTML file opened from the disk with a Flask weather page, on three points: what the page source contains, who does the work, and what must be running.

9.SA12. Three kinds of files belong in `.gitignore`. Name the three kinds, and for each give one example from a Flask project.

9.SA13. A Flask page runs a fresh simulation of a coffee shop's morning on every visit and shows the result. A colleague proposes running the simulation once a day, saving the result as a file, and publishing it as a static site "so no server has to run." What does the page lose and what does it gain?

Chapter 10: How Web Requests Work

Easy

10.MC4. Which four-word rule says when a request should be GET and when POST?

(a) GET sends; POST receives

(b) GET shows; POST changes

(c) GET is safe; POST is slow

(d) GET hides; POST shows

10.MC5. Where is a cookie stored?

(a) on the server

(b) in the browser

(c) in `app.py`

(d) in the URL

10.SA3. Which part of a URL does Flask's `@app.route` line match?

10.SA4. In a query such as `?q=weather&hl=en`, what does the `&` do?

Medium

10.SA6. Why is a query (the part of a URL after `?`) never used for a password? Name three places where a query is visible.

10.SA7. A Flask route that changes something on the server reads its data from `request.form`. What does a route that only shows something read, and from which part of the URL does that come?

10.MC7. A Flask server is running on port 5000 and a visitor opens `http://127.0.0.1/zip`. The connection is refused. Which port did the browser try?

(a) 5000

(b) 80

(c) 443

(d) none; the address is invalid

Hard

10.SA1. A photo app deletes a photo when someone opens `/delete?photo=7`. Name one way the photo could be deleted without anyone clicking a button.

10.SA2. A site remembers that a visitor has paid by setting a cookie `paid=yes`. Why is that unsafe?

10.SA8. A Flask server has a route `/report` that accepts only POST. Its terminal shows:

```
"POST /report HTTP/1.1" 200 -
"POST /report HTTP/1.1" 200 -
"GET /report HTTP/1.1" 405 -
```

The first line is a visitor sending a report. What happened in the second and third?

10.SA9. A weather site remembers the last zip code a visitor looked up in a cookie `last_zip`. In the developer tools, a visitor changes it from `95053` to `hello` and reloads. What should a well-built server do with it, and how should a server regard any cookie?

10.SA10. Two people share one laptop and one browser. One looks up zip code 94305 on a weather site that remembers the last zip in a cookie; an hour later the other opens the zip page and finds 94305 in the box. Is that a bug?

Extra hard

10.SA11. `app.py` has `@app.route("/login", methods=["GET", "POST"])` while `/report` has `methods=["POST"]` alone. A teammate says the login route is a bug because signing in must be POST only. Are they right?

10.SA12. A weather site has a form where the visitor types a zip code; the form uses `method="get"` and the route reads the zip from `request.args`. A teammate changes the form to `method="post"`. What else must change in `app.py`, and what happens to a link such as `/zip?zip=94305` that someone bookmarked last week?

10.MC9. A route that deletes an order reads the order number from `request.args`. What is the bug?

(a) nothing; it works

(b) a change should be a POST reading `request.form`, not a GET reading the query

(c) it should read a cookie instead

(d) the route needs `https`

10.SA14. An order page's address is `https://shop.example.com/orders?customer=1042&email=ana@example.` and support staff like pasting it into emails. What is wrong with it, and what must the server know for a safer address such as `/orders` to work?

Chapter 11: Adding Accounts and a Database

Easy

11.MC3. SQLite stores all of an application's data in:

(a) a separate database server

(b) a single file in the project folder

(c) the browser

(d) `app.py`

11.MC4. What is the name of the cookie Flask uses to keep a user logged in?

(a) `last_zip`

(b) `user`

(c) `session`

(d) `password_hash`

11.MC5. A Flask sign-up app keeps its users, their password hashes, and their bookings in a SQLite file `signup.db`. Which line belongs in `.gitignore`, and why?

(a) `app.py`, because it holds the secret key

(b) `signup.db`, because it holds every user and password hash

(c) `templates/`, because HTML is generated

(d) `requirements.txt`, because it is specific to this computer

11.SA3. Why can a well-built site help you reset a forgotten password but not email you the old one?

Medium

11.SA1. After a booking form is submitted with POST, the server answers with a redirect to the page that shows the bookings instead of with that page itself. Why?

11.SA6. A Flask app keeps logins in a signed `session` cookie. What does the *Log out* button actually do, on the browser's side and on the server's side?

Hard

11.MC7. A Flask app keeps logins in a signed `session` cookie. Someone copies ana's cookie. Ana then logs out. Can the copy still be used?

(a) no; logging out invalidates it

(b) yes; the server keeps no record of logins, so the copy stays valid

(c) only until the server restarts

(d) only from the same laptop

11.SA7. A sign-up app: users register with a password, log in, and book time slots. Users and bookings are in a SQLite file `signup.db`; Flask keeps logins in a signed `session` cookie; the secret key that signs it is written in `app.py`, and `app.py` is on GitHub. For each fact, say where it is kept: ana's password hash; that ana is logged in right now; that Thursday at 3:00 is taken; the secret key. Which of the four is on GitHub, and should it be?

11.MC8. A `bookings` table stores `user_id`, not the username. Ana renames her account. What happens to her booking row?

(a) it is deleted

(b) it must be updated by hand

(c) nothing; it still refers to the same user

(d) it now shows as taken by nobody

11.SA8. In your Flask app, deleting the `session` cookie logs you out, but restarting the server does not. A teammate's app, built from a similar prompt, logs everyone out at every restart. What must be different about their secret key?

11.SA9. A weather app keeps the last zip code typed in a plain cookie `last_zip`, which the server accepts if it looks like a zip code. A sign-up app keeps the logged-in user's ID in a *signed* cookie. Why does the second need a signature when the first does not?

Extra hard

11.SA10. Two students click *Book* on the same slot in the same second. Where is it decided who gets it: in the page, in the route, or in the table?

11.SA11. Ben constructs a cancel request by hand for ana's slot, bypassing the page. Why is hiding the *Cancel* button not enough, and what must the server do?

11.SA12. `app.py` contains `app.config["SECRET_KEY"] = "dev-change-me"` and is on GitHub. What can anyone who reads the repository do?

11.SA13. With the server stopped, you delete `signup.db` and start it again; the app creates a new, empty database. Ana's browser still holds a validly signed `session` cookie saying `user_id` 1. Is the signature still valid? Is ana still a user?

11.SA14. A sign-up app's rule: "the first user who registered is the administrator." Why is that fine on your laptop and wrong for a public site?

Chapter 12: Deployment

Easy

12.MC4. `gcloud config set project ID` does what?

(a) creates a new project

(b) tells `gcloud` which project the following commands refer to

(c) deploys the project

(d) sets the region

12.MC5. A deployed Cloud Run service that nobody visits for a week costs:

(a) the hourly rate for one machine

(b) nothing; Cloud Run stops the copy and the charge stops with it

(c) a flat monthly fee

(d) the price of the container image per day

12.MC6. In `gcloud run services list`, which column tells you who performed the latest deploy?

(a) `URL`

(b) `REGION`

(c) `LAST DEPLOYED BY`

(d) `OWNER`

12.SA3. Why is a Cloud Run service `https` from the first minute, with nothing for you to set up?

Medium

12.SA4. Does deploying with `gcloud run deploy --source .` require that your changes be committed or pushed to GitHub? What exactly is sent?

12.SA5. The deploy asks *Allow unauthenticated invocations?* What does "unauthenticated" mean here, and why is the answer `y` for a sign-up sheet that has its own accounts?

12.SA6. What is a container image, in one sentence?

Hard

12.SA1. On Google Cloud, a teammate with the *Editor* role deploys. The build succeeds and a revision is created, but the step that makes the service public fails. Why?

12.SA8. A teammate deploys from a clone they last pulled a week ago. What is online now, and what should the team's rule be?

12.SA9. `gcloud run services describe` shows `Scaling: Auto (Min: 0, Max: 100)`. What does `Max` mean for an application that keeps its data in a file inside the running copy?

12.MC9. `app.py` ends with a line that starts Flask with `debug=True`. On Cloud Run, is that line running?

(a) yes, and it is harmless

(b) no; `gunicorn` starts the `app` object without running that line, but the line should still go

(c) yes, and it exposes the server

(d) only on the first revision

12.SA10. On your laptop the server starts with `flask --app app run`; on Cloud Run it starts with `gunicorn`, which is not in `requirements.txt`. Where did `gunicorn` come from?

Extra hard

12.SA2. In two sentences, describe a cross-site request forgery against a `/cancel` route, and name the usual fix.

12.SA11. Name three events after which a database file kept inside the running copy of a Cloud Run service is gone.

12.SA12. A sign-up sheet used by 200 students runs on Cloud Run and keeps its data in a SQLite file inside the running copy. Its rule is "the first user who registered is the administrator." What does that rule mean after every redeploy, and what kind of fix does it need?

12.SA13. An AI assistant warned that marking the session cookie `Secure` would make login "appear to fail" on your laptop over plain `http`. You try it in Chrome at `http://127.0.0.1:5000` and login works. What does the warning get right, and what is the lesson?

12.MC10. A group shares one Google password and everyone deploys as the owner. What does `LAST DEPLOYED BY` then tell you?

(a) who really deployed

(b) nothing useful; every deploy names the same account

(c) the IP address of the laptop

(d) the Git author of the last commit

12.SA14. A deployed sign-up app has two problems: its secret key is a fixed string written in `app.py`, which is on GitHub; and its administrator is whoever registered first. Why can neither be fixed by a code change alone? What do both need?

Answers

Chapter 2: Working in the Terminal

2.MC4. (c). *Section 2.2.*

2.SA3. Returns you to your home folder. *Section 2.2.*

2.SA4. `Control+C`, and it is `Control` even on a Mac. *Section 2.6.*

2.MC7. (c). Paths beginning with `/` or `~` are absolute; the others are relative to the current folder. *Section 2.2.*

2.SA1. Two folders, `my` and `notes`. The space separates arguments, so `mkdir` received two names. *Section 2.2.*

2.SA2. Without `-r`, `rm` deletes only files. `rm -r my_notes` deletes the folder and everything in it. *Section 2.4.*

2.SA5. It is overwritten by the contents of `notes.txt`, without any warning. *Section 2.4.*

2.SA6. Two lines: `b` then `c`. The second `>` replaced `a`; `>>` appended. *Section 2.3.*

2.SA7. `a.txt` and `b.txt` are deleted; `old` remains, and the terminal prints `rm: old: is a directory`. *Sections 2.4, 2.5.*

2.SA8. The size is `0` bytes: the file is empty. `cat week4.txt` settles it. *Sections 2.2, 2.3.*

2.SA10. No: a command that fails changes nothing. The silence of the first means it succeeded; commands that succeed are usually silent. *Section 2.2.*

2.SA11. No. `mkdir` created a folder `archive`; `cp` copied the three files into it; `rm` deleted the three originals, permanently. The copies in `archive` are untouched, so the notes are not gone. *Sections 2.4, 2.5.*

2.SA12. `cat` cannot print it: the leading `d` means `notes.txt` is a folder, not a file. *Sections 2.2, 2.3.*

2.SA13. `ls *.log`, with the same pattern. It shows exactly which files the wildcard matches before they are deleted for good. *Section 2.5.*

Chapter 3: Building Your First Web App

3.MC3. (c). HTML is the structure of the page and CSS its appearance; neither one describes what happens when the user acts. *Chapter introduction.*

3.MC5. (b). *Section 3.6.*

3.MC6. (b). *Section 3.1.*

3.SA3. It cannot pass information from one visitor to another (every copy of the page is isolated); it cannot keep anything secret from the visitor (the whole program is on the visitor's computer); and it cannot use any information it did not bring with it (another visitor's data, a list of sign-ups, today's weather). Accept any wording that names these. *Section 3.6.*

3.MC7. (b). Each tab runs its own copy of the program with its own tally, starting from zero. *Section 3.5.*

3.SA5. It needs a back end: items added by one visitor must reach the others, and a page running alone in a browser cannot pass information between visitors. *Section 3.6.*

3.MC8. (b). To mark the answers, the page must contain them; a page running alone in a browser cannot hide anything from the visitor. *Sections 3.4, 3.6.*

3.SA6. 4 in the first tab; 3 in the second. Each tab starts from the file's line and keeps its own count. *Section 3.5.*

3.SA7. Where do the sign-ups go (who sees the list)? A sign-up sheet must pass information from one visitor to the others, and a page running alone in a browser cannot. *Section 3.6.*

3.SA8. No: nothing passes between visitors, nothing is hidden, nothing is fetched. Yes, once the gallery is added: drawings from one visitor must reach the others. *Section 3.6.*

3.SA9. It has not tested the game, and it added a feature nobody requested. Checking is your job, not the assistant's. *Section 3.2.*

3.SA10. The organizer and email are doing the sharing by hand. An application that needs to pass information between visitors needs a back end; the plan has simply replaced it with a person. *Section 3.6.*

3.MC9. (b). The source is the entire program the browser received; whatever the page checks is in it. *Section 3.4.*

Chapter 4: Tracking Changes with Git

4.MC4. (b). It is the place the project was cloned from. *Section 4.1.*

4.MC5. (b). *Section 4.3.*

4.SA1. Working folder → `git add` → staging area → `git commit` → local repository → `git push` → GitHub. *Section 4.1.*

4.SA3. In the hidden folder `.git` inside the working folder. No; only Git writes to it. *Section 4.1.*

4.MC6. (c). `-am` stages only tracked files; a new file needs `git add` first. (Ignored files are also skipped, but that is `.gitignore`'s doing, not `-am`'s.) *Section 4.2.*

4.MC7. (c). It adds a new commit that reverses the change; nothing is erased, and the history on GitHub stays consistent with yours. *Section 4.8.*

4.SA4. In the local repository on your laptop (and reflected in the working folder); not on GitHub. `git push` sends them. *Section 4.4.*

4.SA5. It copies `index.html` from the most recent commit back into the working folder, discarding every change to the file since that commit. *Section 4.7.*

4.MC8. (b). This is why `reset` should not be used on pushed commits; `revert` is the safe tool. *Section 4.8.*

4.SA7. `git add index.html`, then `git commit -m "…"`. `-am` stages every tracked file that changed, so the typo fix would come along. *Section 4.2.*

4.SA8. The commit's seven-character code, which identifies this version. The local repository on your laptop; GitHub does not have it yet. *Section 4.4.*

4.SA9. No: `restore` reaches only the most recent commit, which now holds the empty file. `git revert --no-edit HEAD` adds a commit that puts the previous contents back. *Sections 4.7, 4.8.*

4.SA10. `debug.log` under *Untracked files*: nothing tells their Git to ignore it. They may `git add .` and commit it. `.gitignore` is committed precisely so that everyone ignores the same files. *Section 4.9.*

4.SA11. Only in the local repository (and the working folder) on the dead laptop. No: GitHub has only the commits that were pushed. *Sections 4.1, 4.5.*

4.SA12. No. `restore` acts on the one file named; `README.md` was already modified and is untouched. *Section 4.7.*

Chapter 5: Working with an AI Coding Assistant

5.MC1. (d). A prompt states the stack, one feature, where the files go, and that you want complete files. *Section 5.2.*

5.MC3. (d). Assistants write untested code readily; the quiet mistake is that they describe untested work as finished. *Section 5.1.*

5.MC4. (b). A new conversation knows only what is in the folder. *Section 5.6.*

5.SA3. The files (which exist, with which names, where); the addresses (anything beginning with `http`); your own rules (the numbers and words from your prompt). *Section 5.3.*

5.SA4. Every time something works, before asking the assistant for anything else; a few times an hour. *Section 5.6.*

5.MC6. (b). Loading something from the internet is a decision about where your page connects, and it is yours to make. *Section 5.5.*

5.MC7. (b). One line, then compare the explanation with what you saw the page do. *Section 5.4.*

5.SA5. It made a decision about the shape of your project that you did not ask for. You noticed by checking the files: which exist, with which names. *Section 5.3.*

5.SA6. Every untouched row counts as an A with 3 credits, so the GPA is 4.00 before you type. The sentence is true, but no row is ever blank, so the rule it describes never applies. The assistant made a decision (pre-filled rows) that you did not ask for. *Section 5.1.*

5.SA7. The assistant: it silently drops the row. Whether the page should show an error instead is your decision, not the assistant's; you never specified it. *Section 5.4.*

5.SA8. A commit of the working version. `git restore index.html`. *Section 5.6.*

5.SA9. A button built to send your grades to a destination nobody has chosen yet. The assistant raised the question; only you could say no. *Section 5.5.*

5.SA10. No. 3.69 = 33.2 / 9 is the A and B+ alone (the C row was dropped); subtracting would give 29.2 / 7 = 4.17. One of the two is wrong, and you have found something to fix. *Section 5.4.*

5.SA11. Twelve features produce twelve times the code and one reply claiming all twelve work; you can check one feature in a minute and will check none of twelve properly. One feature per prompt, and a commit after each. *Sections 5.2, 5.6.*

5.SA12. Search the file for one of your scale's values (such as `3.7`) and compare with your prompt. `git restore index.html` if the working version was committed; otherwise ask the assistant to put your scale back and check it again. *Sections 5.3, 5.6.*

Chapter 6: Branches and Collaboration with Git

6.MC4. (b). The capital `-D` is required so that unmerged work cannot be discarded by accident. *Section 6.4.*

6.SA3. Fetches the commits on GitHub that your local repository does not have and adds them to your branch, updating the working folder. *Section 6.5.*

6.SA4. A merge where nobody committed to `main` while the branch was being worked on, so Git only moves `main` forward to the branch's latest commit; no new commit is made. *Section 6.3.*

6.MC2. (b). *Section 6.5.*

6.SA6. Yes. You merge into the branch you are on, so `main`'s new commits are brought into the feature branch; `main` itself is unchanged. *Section 6.3.*

6.SA7. No. They remain as part of `main`; only the name is removed. *Section 6.3.*

6.SA1. Each side has a commit the other lacks, and Git has not been told how to combine them (`pull.rebase` is not set). Nothing was lost. *Section 6.6.*

6.SA2. No. A branch merged while `main` had no new commits of its own is merged by moving `main` forward, and leaves no fork in the graph. *Section 6.3.*

6.SA8. After a fast-forward, both branches point at the same commit. Only the label `computer-player` disappears; the commit itself is unchanged. *Section 6.3.*

6.SA9. They committed but never pushed; or they pushed to a different branch (or a different repository). *Sections 6.5, 4.5.*

6.MC7. (b). A new local branch has never been pushed, so there is no `origin/…` to be up to date with. *Sections 6.1, 6.5.*

6.SA10. Both sides had moved: the laptop and GitHub each had a commit the other lacked, so Git could not simply move forward. Nothing was lost; Git refused to let one side overwrite the other. *Section 6.6.*

6.SA11. They are two separate local repositories that only meet on GitHub. The first must push and the second must pull. *Section 6.5.*

6.SA12. Whatever half-tested work was committed last. With a branch, `main` holds only the working version while new work is written and checked, and takes it in only once it works. *Sections 6.3, 6.4.*

Chapter 7: Collaborating on GitHub

7.MC4. (b). *Section 7.1.*

7.SA3. No; approval must come from someone else. *Section 7.2.*

7.SA1. Does it do what the title says, and only that? Was it checked? Does the team want it? *Section 7.2.*

7.MC6. (c). *Section 7.3.*

7.SA4. A pull request for the branch, and its merge into `main`. A pushed branch is not in `main`. *Sections 7.1, 7.3.*

7.SA5. The diff shows what changed (the first question). It cannot say how the change was checked, or whether the team wants it. *Section 7.2.*

7.SA6. It points to one line, says what is wrong, and suggests what to do. *Section 7.2.*

7.MC7. (b). You move it to a branch and propose it from there. *Section 7.6.*

7.SA8. That it cannot be reviewed as is: ask for a title and description saying what it does and how it was checked, and for it to be split into pull requests that each do one thing. *Section 7.2.*

7.SA9. It is a statement about the code (no bug found, the counter counts correctly). It cannot say whether the feature belongs on the page or where; that is a product decision only a person makes. *Section 7.2.*

7.SA10. `git commit --no-edit` to create the merge commit. Run the application first: two changes that each work alone can break when combined. *Section 7.5.*

7.SA11. A change reached `main` that nobody but its author looked at, with no pull request recording why it was made or who agreed. The record now has a gap. *Sections 7.3, 7.6.*

7.SA12. The rule exists so that someone else inspects every change before it reaches `main`. The record would show a review that never happened. (Accept any answer that names the purpose and rejects the fake approval.) *Sections 7.2, 7.6.*

7.SA13. Fix it on a branch and propose it with a pull request, like any change; the fix is reviewed, and the pull request records what broke, what fixed it, and who agreed. *Sections 7.1, 7.3.*

7.MC8. (b). *Section 7.4.*

Chapter 8: Python Packages and Virtual Environments

8.MC3. (b). *Section 8.1.*

8.MC5. (b). *Section 8.4.*

8.SA3. That the virtual environment is active in this terminal: `python` and `pip` now refer to the copies in `.venv`, and anything installed goes there. *Section 8.2.*

8.SA4. Create the environment (`python3 -m venv .venv`), activate it (`source .venv/bin/activate`), install from the recipe (`pip install -r requirements.txt`). *Section 8.5.*

8.SA2. Flask's authors, who declare the packages Flask depends on; you are trusting their choices too. *Section 8.3.*

8.SA5. One shared Python serves every project; one project's version of a package can break another's, and nothing records which project needed what. *Section 8.2.*

8.SA6. It pins the package to the exact version you tested with, so another computer builds the same environment rather than a mix of versions. *Section 8.4.*

8.SA1. It is generated (it can be rebuilt from `requirements.txt`) and large, and it is specific to one computer and Python version. *Sections 8.2, 8.5.*

8.MC7. (b). The recipe records what was installed in the active environment at the moment it was written. *Section 8.4.*

8.SA8. The `coffee` project's, not `coffee-lab`'s: the activation followed the terminal. Deactivate, or activate `coffee-lab`'s own `.venv`. *Section 8.2.*

8.SA9. Outside an environment a Mac has no `python` command, only the laptop's own `python3`; inside `.venv`, `python` exists and points into the project. *Sections 8.2, 8.4.*

8.SA10. An updated `requirements.txt` in the same pull request. Otherwise every other computer (and the server) that rebuilds from the recipe fails with `ModuleNotFoundError`. *Sections 8.4, 8.5.*

8.SA11. Reproducibility: each computer builds whatever versions exist on the day it installs, so "it works on my machine" returns. *Section 8.4.*

8.SA12. Its Python version (or platform) is one for which that exact numpy version is not published. Run `python3 --version` first. *Sections 8.1, 8.4.*

Chapter 9: Building Your First Flask Server

9.MC4. (b). *Section 9.2.*

9.MC5. (c). *Section 9.4.*

9.MC6. (b). Both are generated and specific to this computer. *Section 9.8.*

9.SA3. `templates/`; the server fills them (via `render_template`) before sending plain HTML. *Section 9.7.*

9.SA2. Another server is already listening on port 5000, usually one started earlier and not stopped. Check the other terminal tabs and stop it with `Control+C`. (On a Mac, AirPlay Receiver can also hold the port.) *Section 9.1.*

9.SA4. The wording is in `templates/`; the wrong result is in `app.py`. *Section 9.7.*

9.SA6. Python generates it (prepared copies of the program) and rebuilds it whenever needed; it was written by no person and differs between Python versions. *Sections 9.1, 9.8.*

9.SA1. 1,000. The service may limit or refuse that many requests from one source, and the visitors then see an error. *Section 9.2.*

9.SA7. `99999` (404: the service said no such zip). `1234` was rejected by the server as malformed (400) before any lookup. *Section 9.4.*

9.SA8. The assistant added one unpinned name; `pip freeze` records every package actually installed, including matplotlib's dependencies, each at its exact version. *Section 9.5; chapter 8.*

9.SA9. The server (with a Python library such as matplotlib), as a finished image written out as text inside the page. The browser only displayed it. *Section 9.5.*

9.SA10. `requirements.txt` (the recipe; without it you cannot rebuild the environment) and `.gitignore`. The missing recipe is what prevents running. *Section 9.7.*

9.MC9. (b). *Section 9.8.*

9.SA11. Game: source is the whole program; the browser does the work; nothing needs to run. Weather: source is only the result; the server does the work; the server must be running all the time. *Section 9.6.*

9.SA12. Generated (`__pycache__/`), specific to this computer (`.venv/`), secret (`.env`; also a database file holding users). *Section 9.8.*

9.SA13. It loses the ability to run a new simulation per visit (the server did the work on request); it gains that nothing has to be kept running. Whether that is acceptable depends on whether a new morning per visit is the point. *Section 9.6.*

Chapter 10: How Web Requests Work

10.MC4. (b). *Section 10.2.*

10.MC5. (b). The server reads what the browser brings back. *Section 10.3.*

10.SA3. The path. *Section 10.1.*

10.SA4. Joins several `name=value` pairs. *Section 10.1.*

10.SA6. The address bar, the browser's history, and the server's log, which records every address in full. *Section 10.1.*

10.SA7. `request.args`, the query. *Section 10.2.*

10.MC7. (b). With no port written, `http` means 80, where nothing is listening. *Section 10.1.*

10.SA1. Any program that opens links: a browser prefetching the page, a search crawler, or an email virus scanner. *Section 10.2.*

10.SA2. The cookie is kept in the visitor's browser, and the visitor can create or edit it. *Section 10.3.*

10.SA8. The second is the same visitor reloading and agreeing to resend, so the report was saved twice. The third is someone typing `/report` into the address bar: a GET on a POST-only route, refused. *Section 10.2.*

10.SA9. Ignore it (use it only if it looks like a zip code). A cookie is data received from a stranger. *Section 10.3.*

10.SA10. No. A cookie belongs to one browser on one computer, and to whoever is using it; the server cannot tell the two people apart. *Section 10.3.*

10.SA11. No. GET shows the login form; POST signs in. `/report` needs no GET because the form that sends to it lives on another page. *Section 10.2.*

10.SA12. The `/zip` route must accept POST and read the zip from `request.form` instead of `request.args`. The bookmark no longer produces a result, since the data is no longer in the address. (Looking up weather changes nothing, so GET was the right choice to begin with.) *Section 10.2.*

10.MC9. (b). A GET that deletes is a bug however well it works when you try it. *Section 10.2.*

10.SA14. Anyone who edits the query can see any customer's orders, and the query is in logs and history. For `/orders` alone to work, the server must know who is asking: a cookie that identifies the logged-in user. *Sections 10.1, 10.3.*

Chapter 11: Adding Accounts and a Database

11.MC3. (b). *Section 11.1.*

11.MC4. (c). *Section 11.2.*

11.MC5. (b). A database is generated, specific to this computer, and secret. *Section 11.4.*

11.SA3. It never stored the password, only a hash computed from it, which cannot be run in reverse. *Section 11.3.*

11.SA1. So that reloading repeats a GET of the page, not the POST, and the booking is not submitted twice. *Section 11.5.*

11.SA6. It tells the browser to throw its cookie away. The server keeps no list of who is logged in, so it has nothing to discard. *Section 11.2.*

11.MC7. (b). This is why the cookie must never travel where it can be copied; HTTPS protects it in transit. *Section 11.2.*

11.SA7. `signup.db`; the browser (session cookie); `signup.db`; `app.py`. The secret key is on GitHub, and it should not be. *Sections 11.2, 11.6.*

11.MC8. (c). The ID links the tables, so each fact is kept in one place. *Section 11.6.*

11.SA8. Theirs is generated anew each start (a random value), so old signatures no longer match; yours is a fixed string. *Section 11.2.*

11.SA9. The session cookie makes a far more serious claim, *I am this user*, so the server must be able to detect tampering; a wrong zip code costs nothing. *Section 11.2.*

11.SA10. In the table: a constraint refuses a second row for the same slot. *Section 11.5.*

11.SA11. The page is only what the browser was sent; the request can be made without it. The server must check, on every request, that the booking belongs to the person asking before deleting the row. *Section 11.5.*

11.SA12. Produce a valid signature for any `user_id`, and so present a cookie that says they are any user, including the administrator, without a password. *Section 11.2.*

11.SA13. The signature is valid (same key). But the new database has no row 1, so the server cannot find a user for it; the page cannot show *Logged in as ana*, because the name comes from the table. *Sections 11.2, 11.3.*

11.SA14. On your laptop the first registrant is you; on a public address it is whoever reaches the register page first, and if the database is recreated empty at every redeploy, the race reopens each time. *Sections 11.5, 12.7.*

Chapter 12: Deployment

12.MC4. (b). *Section 12.2.*

12.MC5. (b). *Sections 12.1, 12.5.*

12.MC6. (c). *Section 12.6.*

12.SA3. Google owns the `run.app` name and holds the certificate that proves it. *Section 12.3.*

12.SA4. No. `gcloud` sends the folder as it is on your laptop, committed or not; GitHub plays no part. *Section 12.3.*

12.SA5. Visitors who have not signed in to Google. The sheet has its own accounts and must be open to anyone. *Section 12.3.*

12.SA6. A complete, frozen copy of the application with its Python and its packages, ready to run anywhere. *Section 12.3.*

12.SA1. Making the service public changes who may use it, and Editor lacks that permission (`run.services.setIamPolicy`); Owner has it. *Section 12.6.*

12.SA8. The week-old code, as a new revision; every change since then is gone from the site though still in GitHub. Pull `main` before you deploy, deploy only what is on `main`, and say that you did. *Section 12.6.*

12.SA9. Under load, up to 100 copies may run at once, each with its own filesystem and therefore its own database; they do not share the rows. *Sections 12.5, 12.8.*

12.MC9. (b). *Sections 12.3, 12.7.*

12.SA10. Cloud Build recognized a Flask application and added a production server to the image. *Section 12.3.*

12.SA2. A page on another site contains a hidden form that submits to `/cancel`; the victim's browser sends their login cookie with it, so the server cancels their booking. The fix is a secret token in every real form, which a forged page cannot know. *Section 12.7.*

12.SA11. A redeploy (new revision); Cloud Run stopping an idle copy and starting a fresh one; a second copy starting under load (each has its own file). Also: deleting the service. *Section 12.8.*

12.SA12. Whoever registers first on the new, empty database becomes administrator: a race reopened at every deploy. The fix is an administrator name that comes from outside the code, not a change to the code alone. *Sections 12.7, 12.8.*

12.SA13. A `Secure` cookie is sent only over `https`; Chrome makes an exception for `127.0.0.1` as a trusted address. The lesson: a reasonable claim is still a claim; checking takes a minute and is how you learn which ones hold. *Section 12.7.*

12.MC10. (b). *Section 12.6.*

12.SA14. A place outside the code to keep a value: the secret key, and the administrator's name. Anything written in the code is on GitHub and reset at every deploy. *Section 12.7.*